



FGdiffusion: Large-Scale Knowledge Graph Generation Using a Diffusion Approach

Adrien Bufort, Lionel Tailhardat

► To cite this version:

Adrien Bufort, Lionel Tailhardat. FGdiffusion: Large-Scale Knowledge Graph Generation Using a Diffusion Approach. 2025. <hal-05410352>

HAL Id: hal-05410352

<https://hal.science/hal-05410352v1>

Preprint submitted on 10 Dec 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

FGdiffusion: Large-Scale Knowledge Graph Generation Using a Diffusion Approach^{*}

Adrien Bafort^{1,*}, Lionel Tailhardat^{1,*}

Abstract

Producing synthetic datasets is essential for the development and refinement of AI pipelines, especially in the form of knowledge graphs when decision support tools for a specific application domain rely on the semantic integration of heterogeneous data. In a context where training data is available, deep graph generation algorithms can be considered for this purpose, provided that the generated graphs are appropriately sized and structured. However, current deep learning models primarily focus on adjacency matrices, which pose scalability challenges for generating large graphs. In this paper, we introduce *FGdiffusion*, a method that combines a diffusion model with a flattened graph representation as input and employs ontology-based constraints at inference to generate large-scale, domain-relevant knowledge graphs. Experiments demonstrate that it achieves competitive graph quality and improved computational efficiency compared to state-of-the-art models for schema-less graph generation. It also highlights the simplicity and flexibility of constrained inference for producing knowledge graphs that align with a reference ontology. While further improvements are possible, *FGdiffusion* opens new avenues for synthetic knowledge graph generation.

Keywords

Synthetic Dataset, Graph Generation, Ontology-Based Generation, Knowledge Graphs, Deep Generative Models

1. Introduction

Knowledge graphs (KGs) [1] provide a powerful framework for representing and interrelating information from diverse sources, enabling advanced algorithmic analysis – such as graph traversal, embeddings, automated reasoning, and visualization. The development of AI-driven solutions across many domains depends on the availability of large-scale, semantically rich KGs. However, acquiring such graphs is often challenging due to data scarcity, incompleteness, or confidentiality restrictions. This data bottleneck hinders the development, testing, and validation of new methods, from data architecture design to knowledge management. Therefore, the ability to generate large-scale synthetic knowledge graphs relevant to specific applications is highly valuable.

Deep generative models – transformer architectures, Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), autoregressive models, and diffusion models – are promising solutions, but existing approaches face significant limitations: many current deep learning models for graph generation rely on modeling the graph’s adjacency matrix [2, 3], a representation that scales quadratically with the number of nodes. This poses severe scalability challenges, making it impractical for generating the large graphs, ranging from hundreds to millions of nodes, needed in many real-world applications. For example, transformer models are constrained by computational resources during training, especially the backward pass, thus restricting both the size of the input graphs and the overall model capacity.

In this paper, we deepen the exploration of deep generative models by introducing *FGdiffusion*, a novel approach to knowledge graph generation that overcomes existing scalability limitations. We propose a deep generative model based on discrete diffusion [4, 5] that operates on a more scalable edge list representation of graphs. The proposed approach can be easily implemented within the usual data science ecosystem (e.g. using well-known packages such as `x-transformers`¹ on cloud computing

^{*} Working draft (2025-12-10).

^{*} Corresponding author.

✉ adrien.bafort@gmail.com (A. Bafort); lionel.tailhardat@orange.com (L. Tailhardat)

🆔 0009-0007-6044-9407 (A. Bafort); 0000-0001-5887-899X (L. Tailhardat)



© 2025 Copyright for this paper by its authors.

¹<https://github.com/lucidrains/x-transformers>

with GPU resources), and we demonstrate its effectiveness by evaluating the performance of *FGdiffusion* on various public datasets and comparing it with other state-of-the-art methods.

A key advantage of our diffusion-based framework is its inherent flexibility, which allows for constrained generation during inference. This enables the synthesis of knowledge graphs that not only replicate the global structural properties of the training data but also conform to specific desired characteristics, such as predefined schemas or ontologies (*i.e.* explicit representations of a discourse domain through concepts and relationships) [6] relevant to the application domain.

Overall, we summarize the main contributions of this work as:

- **An adaptation of discrete diffusion algorithm for graph data** with an edge list representation, which bypasses the scalability issues of adjacency matrix-based methods and facilitates the generation of large-scale graphs.
- **An adaptation of the diffusion framework to incorporate knowledge graph constraints**, such as ontologies, during the inference phase, ensuring the generation of structurally and semantically valid KGs.

The remainder of this paper is organized as follows. Section 2 presents some related work. Section 3 gets into details of the *FGdiffusion* approach. Section 4 describes our experiments and evaluations. Finally, Section 5 concludes the paper and discusses some future work.

2. Related Work

Our work lies at the intersection of deep learning, knowledge graphs, and constrained generative modeling. In this section, we review the related work on these three areas, and then we present the positioning of the *FGdiffusion* approach.

Deep learning for knowledge graphs. The synergy between deep learning and knowledge graphs has led to significant advancements in various tasks. Notable examples include knowledge graph completion, which aims to infer missing relationships within a knowledge graph [7], and knowledge graph reasoning, where deep learning models, including those leveraging Large Language Models (LLMs), are used to answer complex queries based on the information stored in KGs [8]. Furthermore, KGs are frequently employed to enhance the performance of LLMs by providing grounded, structured information to improve the accuracy and relevance of their responses [9]. While these works demonstrate the power of deep learning for understanding and utilizing existing knowledge graphs, our focus is on the generation of novel knowledge graphs. Interaction between knowledge graphs and deep learning systems have been fruitful as two separate systems (generally LLMs and KGs). However we are looking at merging those two systems.

Deep graph generation. The field of deep graph generation has witnessed rapid progress, with numerous algorithms developed to generate graphs with desired properties using deep learning techniques [10]. The formal goal of graph generation is to learn a model, parameterized by θ , that can sample new graphs G from a distribution $p_\theta(G)$ that approximates the true data distribution $p_{\text{data}}(G)$ of the training set. Two prominent families of approaches are autoregressive methods and diffusion-based methods. Figure 1 sketches the functioning of each approach. Autoregressive models generate graphs sequentially, adding nodes and edges step by step [11, 12]. In contrast, diffusion models learn how to generate graphs by starting from a noisy (or masked) representation and iteratively refining it through a reverse diffusion process to obtain a valid graph structure [2, 13].

While various neural architectures have been explored for graph processing, recent advancements suggest that transformer-based models adapted for graph data are very good at capturing complex graph structures [14, 15]. Our work builds upon the foundation of diffusion-based graph generation and adapts it specifically for the task of knowledge graph generation.

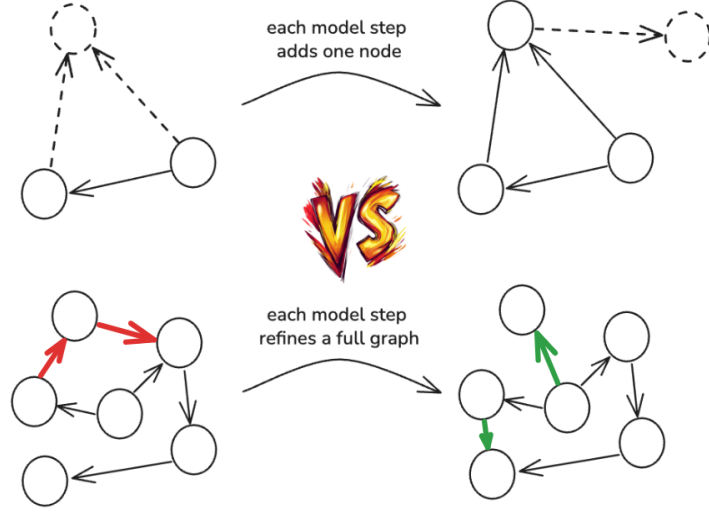


Figure 1: The two main methods for deep graph generation are autoregressive generation (top) and diffusion generation (bottom).

Constrained generative learning. Generating data that adheres to specific constraints is a crucial aspect of many real-world applications of generative learning. This problem has been studied, particularly within the context of diffusion-like models. Approaches to constrained generation can be broadly classified into methods that enforce constraints during the sampling phase and those that incorporate constraints into the model training.

Sampling-based methods aim to guide the generation process towards satisfying the desired constraints without modifying the underlying trained model. For continuous data domains, projection techniques are often used to project noisy intermediate samples onto the valid constraint space during the iterative denoising process [16]. In discrete settings, such as text generation with LLMs, constraints like specific formats or keywords can be enforced by masking the logit outputs during the generation process [17]. In this paper we will take advantage of this method for its simplicity.

Alternatively, constraints can be directly integrated into the training process. This typically involves adding auxiliary loss terms to the training objective that penalize deviations from the desired constraints, forcing the model to learn to generate data that inherently satisfies these conditions [18, 19, 20].

Our positioning. Our work leverages the flexibility of diffusion models and explores methods to incorporate constraints, such as adherence to predefined schemas or ontologies, into the generation process. This allows us to synthesize KGs that are not only structurally sound but also semantically (or ontologically) consistent with a given domain. Also, we make use of the powerful transformer neural architecture and the flattened edge graph representation to enable the generation of large graphs.

3. FGdiffusion: Diffusion-based Graph Generation

In this section, we introduce *FGdiffusion*, an approach combining a discrete diffusion model with a flattened graph representation to address the challenge of generating large-scale domain-relevant knowledge graphs. Its core ideas are as follows.

Diffusion models operate by gradually adding noise or masks to reference datasets in a forward process, and then learn to reverse this process to enable generating new, realistic data samples [21]. These models have shown promise in capturing complex data distributions across various application fields [22, 23]. As a specialized variant, the *discrete diffusion* [4, 24] approach enables using diffusion processes with categorical data. In our case, discrete diffusion captures the underlying structural and semantic patterns present in a set of reference knowledge graphs by gradually adding noise or masks to the graph structure and entity class values. To enhance the scalability of diffusion models, we adopt a

flattened graph representation, which is more favorable than adjacency matrix-based methods [2, 3]. To ensure that the generated knowledge graphs are domain-relevant, we employ *constrained inference* [17] by referring to the ontologies intended to structure the data. That is, we guide the graph generation by progressively unmasking nodes and edges with respect to predefined ontology-based constraints.

Below, we delve into the details of *FGdiffusion*, covering the *input graph flattening* aspect in Section 3.1, *diffusion model training* in Section 3.2, and *graph generation under ontology constraints* in Section 3.3. The evaluation of the approach is presented later in Section 4.

3.1. Flattened Graph Representation as Input Data

In the sequence of data processing steps carried out by *FGdiffusion*, the first involves adapting the input data graphs provided to the pipeline. This adaptation addresses two challenges: the first aims at standardizing the training process by *normalizing the graph model*, especially when dealing with RDF knowledge graphs [25, 26, 27] as input data; the second seeks to improve the scalability of diffusion models by *flattening the graph representation*. Figure 2 outlines these first steps in the context of the training pipeline.

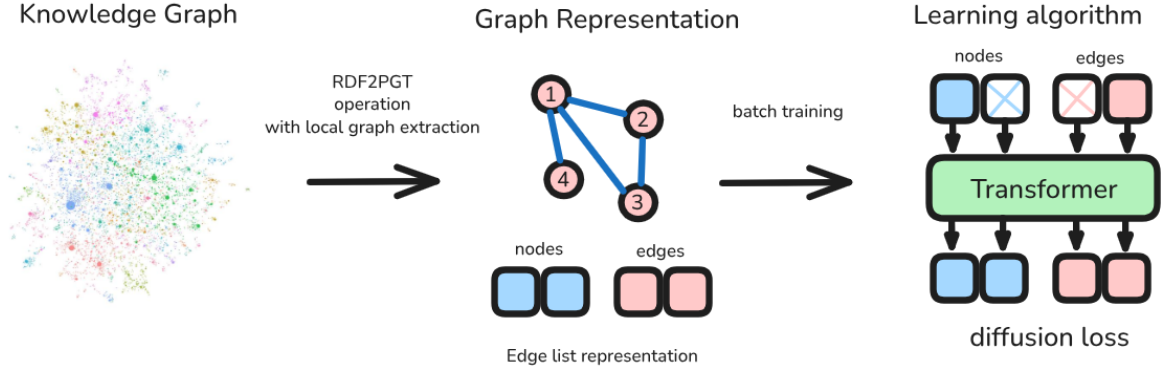


Figure 2: Training pipeline principles.

It is composed of three elements (from left to right): a proper dataset (knowledge graph) with a RDF representation, the transformation of RDF format into a graph format with property graph transformation (PGT) [28, 29] where labels are transformed into graph with node types and edge type with corresponding index value, and finally the proper learning algorithm with a mask / diffusion generative learning setup.

Normalizing the graph model. The key idea is that entity classes will be transformed into node categories and object properties into edges categories (Eq. 1).

$$(NodeClass, attributes = \{(k, v)\}) \xrightarrow{EdgeClass} (NodeClass, attributes = \{(k, v)\}) \quad (1)$$

In the case of RDF knowledge graphs, this normalization step consists in converting the original knowledge graph into a graph representation where nodes and edges possess attributes or categories corresponding to their RDF classes. We utilize the RDF-to-Property Graph (RDF2PG) mapping described in [29] to perform this transformation, excluding datatype properties². Using the terminology of Semantic Web technologies, and more specifically the OWL vocabulary [27], a *NodeClass* is akin to an `owl:Class`, and an *EdgeClass* is akin to an `owl:ObjectProperty`. It is noteworthy to mention that, although *FGdiffusion* can handle RDF knowledge graphs, which is a graph model highly reliant on ontologies, the above-discussed graph model normalization step allows for the generalization of *FGdiffusion*’s use across different KG formalisms; this aspect is demonstrated further below in Section 4 through experiments on “classical” graphs.

²We note that literals are more challenging to generate consistently and leave this for future work.

Local graph extraction. Training a transformer model directly on a large-scale graph is often computationally infeasible due to memory and processing limitations. To circumvent this issue, we employ a stochastic subgraph sampling strategy (Figure 3). For each training step, we extract a smaller, localized subgraph that serves as an input to the model. A neighborhood is then expanded around this seed node through breadth-first sampling to create an induced subgraph (G_{sub}), up to a predefined maximum size (P_{max}).

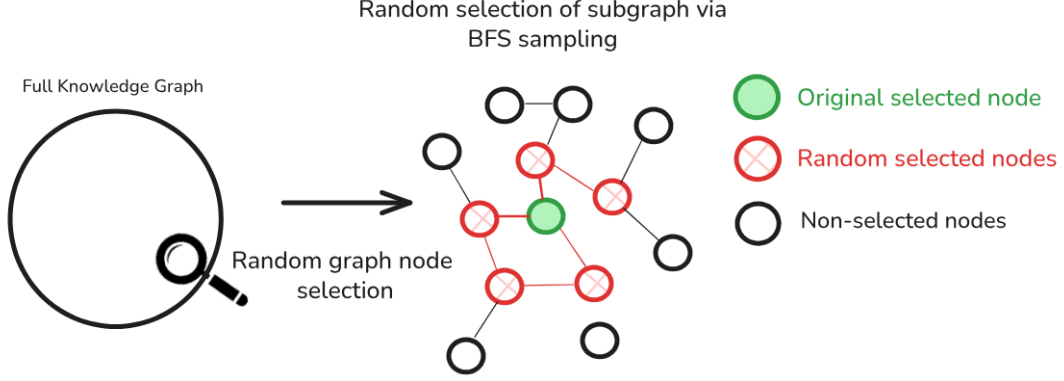


Figure 3: Local graph extraction and sampling principles.

During training, we sample a batch of subgraphs of the original knowledge graph through random node selection and breadth first search (BFS) expansion.

Flattening the graph representation. We posit that an *edge list* representation can significantly reduce the complexity of the generation process compared to models based on adjacency matrix representations, particularly in terms of 1) *memory efficiency*: an adjacency matrix for a graph with n nodes requires $O(n^2)$ space, whereas an edge list requires $O(m)$ space, where m is the number of edges; and 2) *flexibility*: edge lists can easily accommodate graphs with varying densities and sizes, making them suitable for large-scale graphs. Notably, we assume that using an edge list representation enables generative models to be trained on and generate graphs with a larger number of nodes – typically 200+ in our experiments (Section 4), a scale that cannot be achieved using adjacency matrix representations [2, 3].

Therefore, to set up the graph generation model, we consider a flattened representation – which format is inspired by [12] – of the graph data as a combination of labeled nodes and an edge list to serve as input to the forward diffusion process. Essentially, this involves transforming the property graph into a *list of node indices* with their respective classes, such as (Eq. 2):

$$[(0, NodeClass_0), (1, NodeClass_1), (2, NodeClass_2) \dots] \quad (2)$$

and a *list of edges* indicating the nodes they connect along with their classes, such as (Eq. 3):

$$[(1, 2, EdgeClass_0), (1, 3, EdgeClass_1), (1, 4, EdgeClass_2), (2, 3, EdgeClass_3) \dots] \quad (3)$$

3.2. Discrete Diffusion-Based Graph Model Training

The second step of *FGdiffusion* involves training a model for graph generation. To control the size of the dataset relative to computational capabilities for this *graph transformer model training* phase, we leverage the initial phase of *local graph extraction* from Section 3.1. Figure 4 outlines the principles of the transformer model training, and Algorithm 1 provides the details for the entire training sequence.

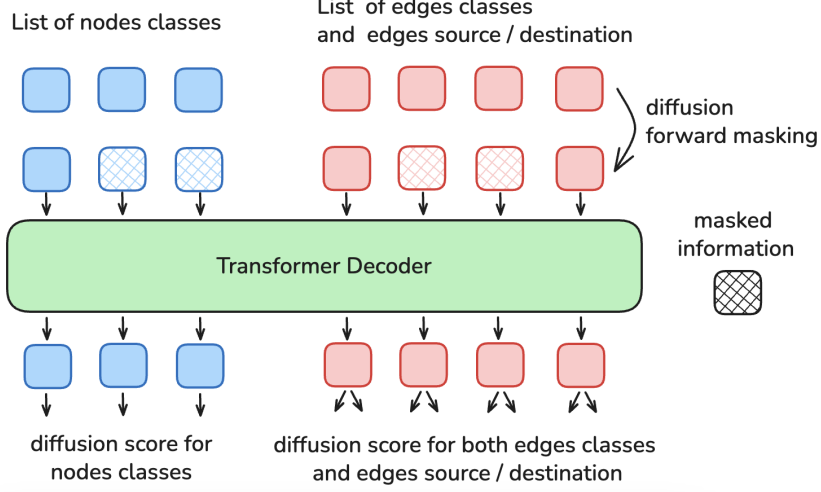


Figure 4: Graph transformer training principles.

We use a flattened graph representation of nodes and edges as input of the transformer setup. We randomly mask some of the token values and then the transformer computes logits for those masked values to estimate the masked piece of the graph. The loss is computed specifically on those masked values.

Discrete diffusion modeling. In this paragraph, we briefly recall the notations and objectives of discrete diffusion modeling and refer the reader to the seminal work in [4] for more details.

The forward diffusion process in a discrete state space $\mathcal{X} = \{1, \dots, N\}$, made of the node indexes and class values in our case, is defined as a continuous-time Markov process. It gradually perturbs the initial data distribution $p_0 = p_{\text{data}}$, transforming it into a simple base distribution p_{base} , as time t approaches infinity. This evolution is governed by a linear ordinary differential equation (ODE, Eq. 4):

$$\frac{dp_t}{dt} = Q_t \cdot p_t \quad (4)$$

where $Q_t \in \mathbb{R}^{N \times N}$ is the diffusion rate matrix at time t . The matrix Q_t has non-negative off-diagonal entries, and its columns sum to zero, which ensures that the total probability mass is conserved over time.

The goal of a generative model is to reverse this noising process: starting from a sample drawn from the base distribution p_{base} at time T , we aim to denoise it to recover a sample from the original data distribution p_0 . This reverse process is also a continuous-time Markov process and has a well-defined time-reversal, described by the ODE in (Eq. 5):

$$\frac{dp_{T-t}}{dt} = \bar{Q}_{T-t} \cdot p_{T-t} \quad (5)$$

The core challenge into obtaining such generative model is to create an artificial neural network that is able to approximate $\bar{Q}$ (Eq. 6) – the so-called “reverse rate matrices”:

$$\bar{Q}_t(y, x) = \frac{p_t(y)}{p_t(x)} Q_t(x, y) \quad (6)$$

to then perform the backward diffusion process using $\bar{Q}^\theta$ (we note θ the neural network parameters). We remark that, in [4], the “score entropy” s_θ (Eq. 7) is utilized as a proxy to approximate $\bar{Q}$:

$$s_\theta(x, t)_y \approx \frac{p_t(y)}{p_t(x)} \quad (7)$$

Graph transformer model training. To implement discrete diffusion modeling, we consider the well-known transformer architecture [30] with an encoder-only transformer. We note that encoder-only

transformers are a standard training architecture within the transformer family and are also found in other deep graph generation models (e.g. G2PTSmall [12], as discussed in Sections 2 and 4.2).

FGdiffusion uses the flattened graph representation from Section 3.1 as input tokens for the transformer. Following the discrete forward diffusion approach in [4], training involves masking a portion of the input tokens and using the transformer to predict scores for the backward diffusion process. Figure 4 illustrates these principles, and Algorithm 1 provides the detailed training setup of our diffusion graph transformer.

Algorithm 1 Discrete Diffusion-Inspired Training for Knowledge Graph Generation

Require: Input Knowledge Graph G (in PGT representation), Transformer model architecture f_0 (uninitialized), Initial model weights θ_{init} (or random initialization), Number of training steps nb_steps , Maximum nodes per subgraph P_{max} , Masking probability p_{mask} (or a masking scheduler/strategy), Learning rate α

Ensure: Output : Trained model parameters θ_{final}

- 1: Initialize model parameters: $\theta \leftarrow \theta_{init}$
 - 2: Initialize Optimizer Opt (e.g., Adam) with learning rate α for parameters θ
 - 3: **for** $step \leftarrow 1$ to nb_steps **do**
 1. *Subgraph Extraction*
 - 4: $N_{center} \leftarrow \text{RandomlySelectNode}(G)$
 - 5: $G_{sub} \leftarrow \text{ExtractNeighborSubgraph}(G, N_{center}, P_{max})$ ▷ Subgraph around N_{center} up to P_{max} nodes
 2. *Flattening Representation*
 - 6: $S_{original} \leftarrow \text{FlattenGraphToSequence}(G_{sub})$ ▷ e.g., sequence of node and edge type tokens
 3. *Masking (Simulating a "diffusion" / noising step)*
 - 7: $S_{masked}, S_{target_score_tokens} \leftarrow \text{ApplyForwardDiffusionMasking}(S_{original}, p_{mask}, t)$ ▷ S_{masked} is $S_{original}$ with tokens replaced by special [MASK] token. $S_{target_score_tokens}$ are the corresponding score.
 4. *Prediction and Loss Computation*
 - 8: $score_{predicted} \leftarrow f(S_{masked}, \theta, t)$ ▷ Model f predicts the original score tokens for the [MASK]ed positions based on S_{masked}
 - 9: $L \leftarrow \text{ComputeLoss}(score_{predicted}, S_{target_score_tokens})$ ▷ e.g., Cross-entropy loss between predicted logits and true target tokens for masked positions
 5. *Model Weight Update*
 - 10: Compute gradients $\nabla_{\theta} L$
 - 11: Update $\theta \leftarrow Opt(\theta, \nabla_{\theta} L)$ ▷ Update model weights using the optimizer
 - 12: **end for**
 - 13: $\theta_{final} \leftarrow \theta$ **return** θ_{final}
-

3.3. Inference Time Ontology Forcing

The third and final step of *FGdiffusion* focuses on utilizing the model developed in Section 3.2 for KG generation. A key challenge is to ensure that the generated KGs are domain-relevant. To address this, we employ a *constrained inference* approach, with *ontology-based constraints*. Below, we further explain these two aspects. Figure 5 outlines the principles of transformer-based constrained inference, and Algorithm 2 provides detailed steps for the entire inference process.

Constrained inference. The inference process begins with a fully masked graph and gradually unmask nodes and edges with the transformer model. Building on this, ensuring domain relevance means using *hiding strategies* [17] during inference (i.e. controlling how the unmasking step is carried out) to force newly created nodes and edges to comply with a set of ontology-based constraints. More precisely, we use a structured generation process by first generating the graph nodes, and then generating the relationships, allowing only those valid with respect to their predicted type and the constraints further discussed below.

Ontology-based constraints. Considering RDFS/OWL ontology axioms as a basis to define constraints, we model constraints as indicator functions³ applied to node features, edge features, or both. To enable such approach, we assume the $\langle \text{subject}, \text{predicate}, \text{object} \rangle$ triples form, from the knowledge

³The indicator function $\mathbf{1}_A$ equals 1 if $x \in A$, and 0 otherwise.

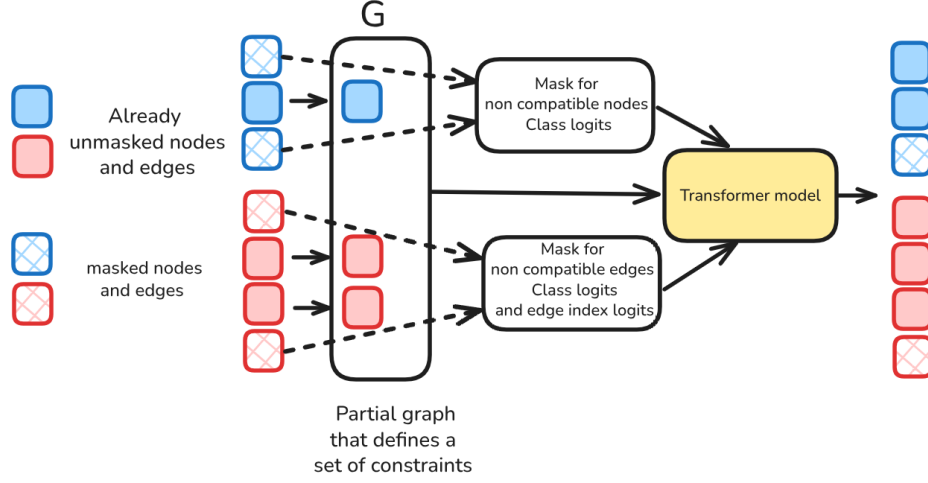


Figure 5: Inference time ontology forcing principles.

To ensure that the generated knowledge graph is domain-relevant, we employ a constrained inference approach, controlling how the demasking step is performed. Demasking relies on a partial graph G composed of already unmasked nodes and edges, along with ontology-based constraints to determine which nodes and edges to unmask.

graph realm, is equivalent to the $node_{source} \xrightarrow{edge} node_{target}$ form seen by the transformer at inference time. In this paper, we consider the following constraints as fundamental; others may be explored later:

- **Property/relation/edge domain and range.** For a given triple, the edge to be unmasked must comply with the domain and range constraints defined within the ontology, *i.e.* for some `owl:ObjectProperty` be coherent with its defined `rdfs:domain` and `rdfs:range`. For example, assuming we are working with the **ORG** [31] ontology – which enables representing organizational structures – unmasking a `org:memberOf` candidate edge involves it connects a `foaf:Agent` node in G with a `org:Organization` node in G .
- **Property/relation/edge maximum cardinality.** For a given $node_{source}$ entity, this constraint restricts the number of times that a specific relation can be used. It is typically set in ontologies using `owl:maxCardinality` in an `owl:Restriction owl:onProperty` construct.
- **Functional property.** We also consider the special case of maximum cardinality set to one, typically implemented with the `owl:FunctionalProperty` construct, which states that a $node_{source}$ entity can have at most one value for this property (*i.e.* for some $node_{source}$, the given $\xrightarrow{edge}$ cannot be unmasked twice).

Algorithm 2 Inference under ontology constraints

Require: Transformer model architecture f , trained model weights θ_{final} , Number of steps nb_steps to generate sample, Maximum nodes in generated graph nbN_{max} , Maximum edges in generated graph nbE_{max} , Ontology constraints $\mathcal{O}$

Ensure: Output: A new generated graph G_{new} which respect ontological constraints.

```
1: Initialize a set of generated nodes  $N_{new} \leftarrow [MASK]*nbN_{max}$  and a set of generated edges  $E_{new} \leftarrow [MASK]*nbE_{max}$   
   Diffusion unmasking loop  
2: for  $i \leftarrow 1$  to  $nb\_steps$  do  
   1. Compute Node and Edge logits  
3:    $t \leftarrow 1 - \frac{i}{nb\_steps}$   
4:    $E_{score}, N_{score} \leftarrow f(E_{new}, N_{new}, \theta_{final}, t)$   
   2. Adding ontology constraints  
5:   Create the transitions matrices  $\bar{Q}_{T-t}^\theta$  from score values.  
6:   for each node and edge that are still masked do  
7:     Apply domain constraints / cardinality / functional to filter valid node and edge types / edge indexes (updating transition matrices with valid transition only)  
8:   end for  
   3. Sample from valid logits  
9:   Sample new nodes and edges from transitions matrices  
10:  Update  $N_{new}$  and  $E_{new}$  with sampled values  
11: end for  
12:  $G_{new} \leftarrow (N_{new}, E_{new})$  return  $G_{new}$ 
```

4. Experiments & Evaluation

In this section, we address the evaluation of the *FGdiffusion* proposal from Section 3, starting with a description of the experimental setup (Section 4.1) and followed by the presentation and analysis of the results (Section 4.2).

4.1. Experimental Setup

Compute environment and model parameters. The environment used for both training and inference is based on an NVIDIA L4 GPU⁴ with 24 GiB of memory, and utilizes NetworkX⁵ v3.4.2 / PyTorch⁶ / PyTorch Lightning⁷ v2.4. For all experiments, we employ a transformer encoder architecture with a hidden (latent) dimension of 384, consisting of 6 attention layers, each with 6 heads, totaling approximately 10 million parameters.

Workflow parameters. Obtaining the experimental results involves the following steps. First, we train a set of models on reference graph datasets (which are described in more detail below), with the number of training steps ranging from 20,000 to 40,000 and a batch size of 128 for each dataset.⁸ We use $P_{max} = 64$ for the local graph extraction step (Section 3.1). Next, with each trained model, we generate a set of 10,000 graphs and measure the inference time. Producing each graph involves 128 sequential iterations of the diffusion model (without optimizations such as using shortcuts [32] or consistency [33]), with each iteration resulting in a subgraph of 64 nodes. Finally, we compute the evaluation metrics (MMD and VUN metrics, as explained below) using both the generated and training datasets.

Datasets. To test and evaluate our approach, we leverage two groups of datasets. First, focusing on *standard graph generation* (i.e. schema-less graphs), we use the following two “classic” datasets, as described in the SPECTRE work by Martinkus *et al.* [34]: 1) the **Planar Graphs** dataset, which consists of 200 distinct graphs (each with 64 nodes); and 2) the **Tree Graphs** dataset, which also contains 200

⁴<https://www.nvidia.com/en-us/data-center/l4/>

⁵<https://networkx.org/>

⁶<https://pytorch.org/>

⁷<https://lightning.ai/pytorch-lightning>

⁸The training duration is approximately 10 h on a L4 GPU.

graphs with either 64 or 100 nodes. Figure 6 provides an example of the contents of these datasets. We use these datasets to evaluate the general performance of the graph diffusion generative algorithm and compare it to state-of-the-art algorithms (see notably Table 2 for the results).

Secondly, focusing on *ontology-related graph generation*, we also consider the following three knowledge graph datasets: **nasa-gesdisc/nasa-eo-knowledge-graph** (NASA KG) [35] interconnects information about satellite datasets, scientific publications, instruments, platforms, projects, data centers, and science keywords; the **Microsoft Academic Graph** (MAG KG) [36] is composed of scientific publication records, citation relationships between those publications, as well as authors, institutions, journals, conferences, and fields of study; the **Computer Science Knowledge Graph** (CS-KG 2.0) [37] is a knowledge graph similar to the MAG KG, structured by the CS-KG ontology⁹, and constructed from research publications in computer science from 2010 to 2022. Table 1 presents the key characteristics of these datasets, including node categories and node-to-node relationship categories (specifically `owl:Class` and `owl:ObjectProperty`, respectively, using the terminology of Semantic Web technologies [27]). Figure 7 illustrates an example of the contents of these datasets, highlighting the class distribution for each node. We use these datasets to evaluate the performance of *FGdiffusion* in constrained inference (see Section 3.3 and Table 4).

Since NASA KG and MAG KG are tabular datasets with side schema descriptions, we transform them directly into property graphs – replacing labels with integers in NetworkX – for processing by *FGdiffusion*. For CS-KG, an RDF dataset, we use the RDF2PG approach from Section 3.1 for processing by *FGdiffusion*.

Table 1

Key characteristics of ontology-related graph generation datasets.

EC: Entity Count (nodes), *CC*: Class Count, *CTop3*: Top-3 classes within EC, *RC*: Relation Count (vertices), *OPC*: Object Property Count, *OPTop3*: Top-3 object properties within RC. *Native format* refers to the dataset’s original, unprocessed form before input into *FGdiffusion*, such as CSV files representing node/edge lists as comma-separated values, or RDF data serialized in Turtle syntax [38].

Dataset Name Domain	EC	CC	CTop3	RC	OPC	OPTop3	Native format
NASA KG [35] <i>Earth science</i>	$\simeq 146K$	7	Dataset DataCenter Instrument	$\simeq 407K$	9	CITES RESEARCH_AREA HAS_DATASET	CSV
MAG KG [36] <i>Scientific publications</i>	$\simeq 2M$	4	Authors Papers FieldsOfStudy	$\simeq 21M$	4	has_topic writes cites	CSV
CS-KG 2.0 [37] <i>Scientific publications</i>	$\simeq 25M$	5	Method OtherEntity Task	$\simeq 67M$	187	:usesMethod skos:broader :usesOtherEntity	RDF/TTL

Evaluation metrics. We adopt the methodology used in the works of Martinkus *et al.* and Chen *et al.* [34, 12] for evaluating graph generation algorithms. For instance, we employ the following groups of evaluation metrics:

- **MMD metrics:** The Maximum Mean Discrepancy (MMD) compares the distributions of graph statistics between generated and test graphs. We focus on node degree (Deg.), clustering coefficient (Clus.), and orbit count (Orbit). The generated graphs produce distributions for these statistics, which are then compared to the data distributions using a distance measure. Overall, lower values indicate better similarity.
- **VUN metrics:** This set includes three metrics. Validity checks whether the generated graphs are truly planar / tree-like or if the generated knowledge graphs respects their related ontologies.

⁹<https://scholkg.kmi.open.ac.uk/cskg/ontology>

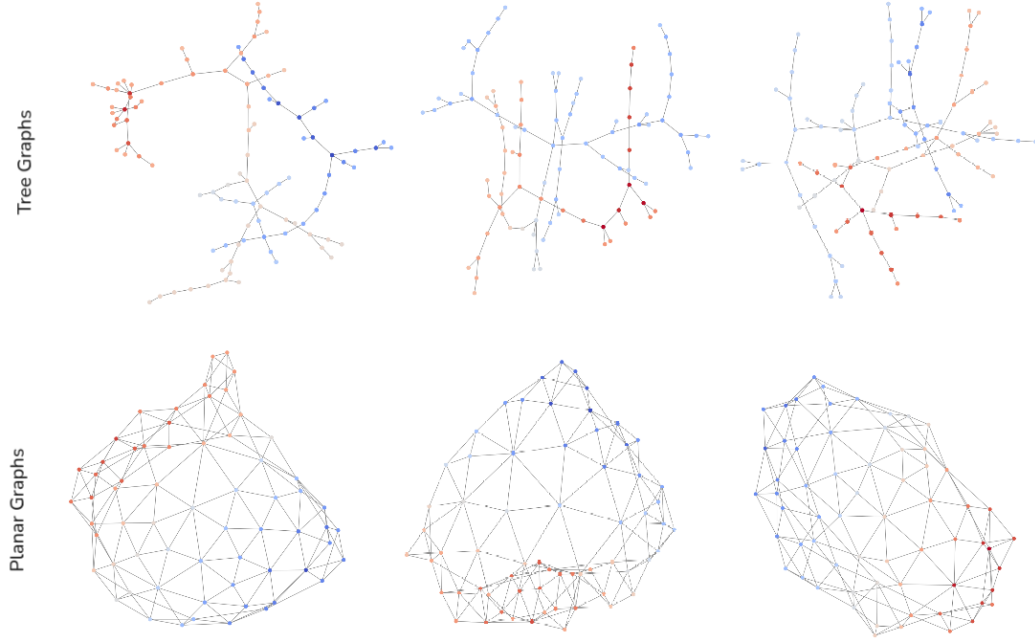


Figure 6: A visualization of the “classics” datasets used: Tree Graphs and Planar Graphs.

The visuals were produced by randomly selecting three graphs from each dataset and then using the NetworkX (<https://networkx.org/>) library’s draw function with the `spring_layout` option (Fruchterman-Reingold force-directed algorithm).

Uniqueness ensures that the generated graphs are diverse enough; otherwise, the generation is not useful. Novelty measures whether the generated graphs are new and not already present in the training set. Overall, higher values for these metrics are better.

4.2. Results

In this section, we first evaluate *FGdiffusion* on standard graph generation, comparing it with state-of-the-art methods in terms of graph quality (Table 2) and computational performance (Table 3). Then, we test *FGdiffusion* on ontology-based graph generation tasks (Table 4). Finally, we discuss the results.

Standard graph generation. Based on the two “classic” datasets from Section 4.1, *FGdiffusion* successfully generates tree and planar graphs, with MMD and VUN metrics that are comparable to those of eight other algorithms considered state-of-the-art in [12], although generally ranking lower. Detailed results are in Table 2.

With the Planar Graphs dataset, *FGdiffusion* ranks 8/9 for Deg@MMD (better than BwR), 9/9 for Clus@MMD (last in the row), 8/9 for Orbit@MMD (better than BwR), and is last overall for the V.U.N. metrics (alongside BwR and GRAN).

For the Tree Graphs dataset, *FGdiffusion* ranks 9/9 for Deg@MMD, tied for top position for Orbit@MMD (with BiGG, DiGress, HSpectre, GEEL, DeFoG, and G2PTsmall), and ranks 7/9 (before GRAN and BwR) for the V.U.N. metrics. Through visual analysis of the generated graphs, we observe that they globally correspond to trees, but occasionally an edge connects two branches here and there, which causes the VUN score to drop to 0 due to non-conformity in its Validity component.

Compute performance. To evaluate *FGdiffusion*’s efficiency, we measured its performance on the aforementioned compute environment (Section 4.1) in terms of *training steps per second* and *inference time per graph* (IT) for the Planar Graphs dataset. Training achieves approximately 1.2 steps per second with a batch size of 128 and a subgraph size of $P_{\max} = 64$, enabling 20,000 to 40,000 steps without

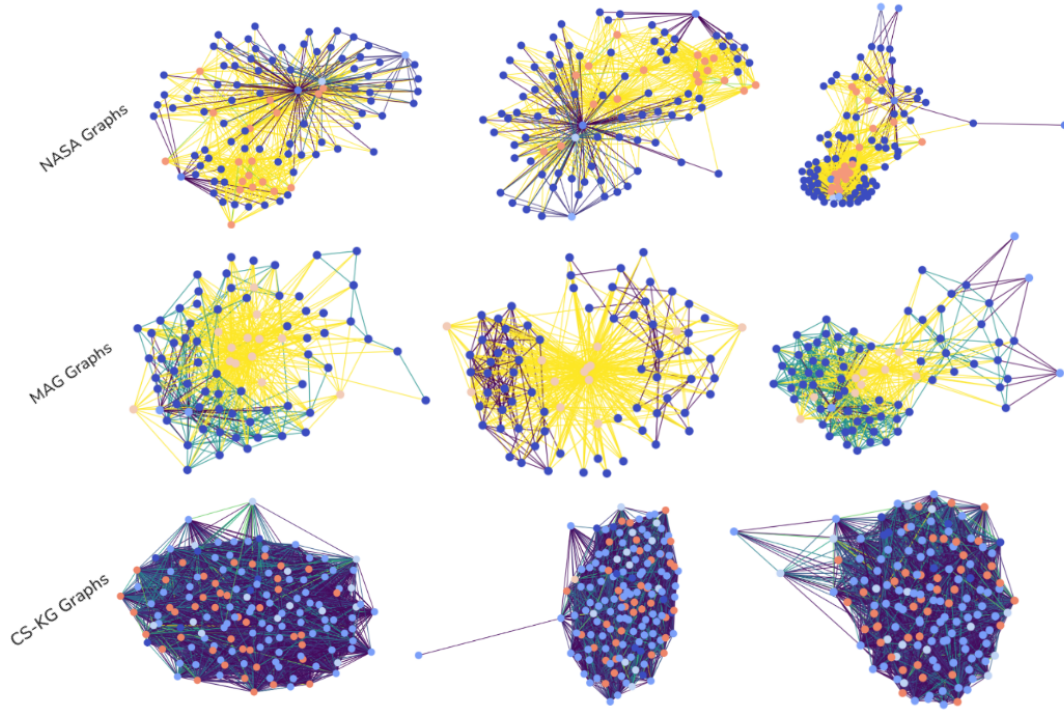


Figure 7: A visualization of the “KG” datasets used: NASA, MAG and CS-KG.

Each node color corresponds to a specific node class and each edge color corresponds to an object property class. The visuals were produced in the same way as for Figure 6.

Table 2

FGdiffusion on the standard graph generation tasks.

We report on the performance of the FGdiffusion approach on the two “classic” datasets and compare it with state-of-the-art graph generation models on the same datasets. Metrics include Degree (Deg.), Clustering (Clus.), and Orbit. Lower is better for these (↓). V.U.N. (Validity, Uniqueness, Novelty) is a percentage, and higher is better (↑). The performance values for algorithms other than FGdiffusion are directly taken from the G2PT paper [12, Table 2]. We also include the models’ architecture and complexity (Arch.) to enable performance comparison from a compute perspective.

Model	Planar				Tree				Arch.
	Deg.↓	Clus.↓	Orbit↓	V.U.N. [%]↑	Deg.↓	Clus.↓	Orbit↓	V.U.N. [%]↑	
GRAN [39]	7.0e-4	4.3e-2	9.0e-4	0.0	1.9e-1	8.0e-3	2.0e-2	0.0	GNN
GEEL [40]	1.0e-3	1.0e-2	1.0e-3	27.5	1.5e-3	0.00	2.0e-4	90.0	LSTM
BiGG [41]	7.0e-4	5.7e-2	3.7e-2	5.0	1.4e-3	0.00	0.00	75.0	TreeLSTM
HSpectre [42]	5.0e-4	6.3e-2	1.7e-3	95.0	1.0e-4	0.00	0.00	100.0	PPGN
BwR [43]	2.3e-2	2.6e-1	5.5e-1	0.0	1.6e-3	1.2e-1	3.0e-4	0.0	RNN
DeFoG [44]	5.0e-4	5.0e-2	6.0e-4	99.5	2.0e-4	0.00	0.00	96.5	Transformer
DiGress [2]	7.0e-4	7.8e-2	7.9e-3	77.5	2.0e-4	0.00	0.00	90.0	Transformer
G2PTsmall [12]	4.7e-3	2.4e-3	0.0	95.0	2.0e-3	0.00	0.00	99.0	Transformer
<i>FGdiffusion</i>	9.7e-3	9.4e-2	4.9e-2	0.0	2.7e-3	0.00	1.8e-1	17.0	Transformer

high resource demands, thanks to the linear scaling of the flattened edge list. The average inference time per graph for the Planar Graphs dataset is $\overline{IT} \approx 1.4s$.

To further evaluate *FGdiffusion*’s efficiency, Table 3 also compares the inference time per graph of state-of-the-art models, including their model types and compute platform specifications to ensure a fair comparison. For instance, we normalize the inference times with the GPU’s theoretical performance in terms of tera floating point operations per second (TFLOPS) and a deep learning performance metrics (DLPerf) [45]. Leveraging GEEL benchmark values [40], TFLOPS data from the TechPowerUp

website^{10,11} and DLPerf scores from the vast.ai website¹², we observe that *FGdiffusion* is competitive with autoregressive models like GEEL ($IT_{FGdiffusion} \approx 6 \times IT_{GEEL}$) and faster than diffusion baselines like DiGress, owing to its discrete diffusion process.

Table 3

Inference time per graph for the Planar dataset.

We report on the inference (IT) per graph for FGdiffusion and state-of-the-art models, highlighting the model type and compute platform used. To ensure fair inference time comparison, we also report on the GPU theoretical performance in terms of tera floating point operations per second (TFLOPS) and deep learning performance metrics (DLPerf) [45], and normalize the inference time with respect to both as IT.TFLOPS and IT.DLPerf values. Lower is better for IT.TFLOPS and IT.DLPerf ($\downarrow$). The inference time values for algorithms other than FGdiffusion are directly taken from the GEEL [40] paper. TFLOPS data are from the techpowerup.com website, DLPerf scores are from the vast.ai website.

Model	Type	IT [s]	GPU	TFLOPS FP32	DLPerf	IT.TFLOPS $\downarrow$	IT.DLPerf $\downarrow$
GEEL [40]	Auto. Reg.	0.25	GTX 1080 Ti	11.34	5.3	2.83	1.32
BiGC [41]	Auto. Reg.	0.61	GTX 1080 Ti	11.34	5.3	6.91	3.23
GRAN [39]	Auto. Reg.	0.66	GTX 1080 Ti	11.34	5.3	7.48	3.49
DiGress [2]	Diffusion	12.09	GTX 1080 Ti	11.34	5.3	137.10	64.07
GDSS [46]	Diffusion	15.25	GTX 1080 Ti	11.34	5.3	172.93	80.82
<i>FGdiffusion</i>	Diffusion	1.40	L4	30.29	25.8	42.40	36.12

Ontology-related graph generation. For the KG datasets, we report the MMD and VUN metrics in Table 4 for running *FGdiffusion* with G_{sub} of size $P_{max} = 64$ (i.e. for local graph extraction and for generating graphs of the same size, as discussed in Section 3.1). Overall, we observe that applying forcing (i.e. activating ontology-based constraints at inference time, as discussed in Section 3.3) significantly increases the VUN measure of the resulting graphs, with an average 34.3 points increase ($\sigma = 31$). At the same time, forcing decreases the distance-based performance metrics of graph quality, with an average $-1.2e-2$ Deg@MMD gain ($\sigma = 1.4e-2$), and an average $-1.5e-2$ Clus@MMD gain ($\sigma = 1.0e-1$). Regarding Orb@MMD values, they are about three orders of magnitude higher than the highest values in Table 2 for the “classical” datasets. Since the same calculation code was used for both KG and classical datasets, we believe this difference may be due to the topology of the KGs, which can cause more errors on this metric.

Table 4

FGdiffusion on the knowledge graph generation tasks.

We report on the performance of the *emphFGdiffusion* approach, with and without the application of ontology constraints, for G_{sub} of size $P_{max} = 64$ on the three “ontology-related graph generation” datasets. Metrics include Degree (Deg.), Clustering (Clus.), and Orbit. Lower is better for these ($\downarrow$). V.U.N. (Validity, Uniqueness, Novelty) is a percentage, and higher is better ($\uparrow$).

Dataset	Model	Deg $\downarrow$	Clus $\downarrow$	Orb $\downarrow$	V.U.N. [%] $\uparrow$
<i>Nasa KG</i>	FGdiffusion	5.5e-3	2.0e-1	48.0	10.0
	FGdiffusion, with constraints	5.3e-1	1.1e-1	51.0	40.0
<i>MAG KG</i>	FGdiffusion	1.3e-3	6.8e-2	12.5	0.0
	FGdiffusion, with constraints	8.3e-3	1.1e-1	13.2	68.0
<i>CS-KG</i>	FGdiffusion	0.0	2.4e-1	24.7	0.0
	FGdiffusion, with constraints	3.0e-2	3.5e-1	28.2	5.0

Discussion. Overall, our experiments confirm that combining flattened graph representations, diffusion modeling, and guided unmasking effectively tackles scaling and domain relevance in knowledge

¹⁰<https://www.techpowerup.com/gpu-specs/geforce-gtx-1080-ti.c2877>

¹¹<https://www.techpowerup.com/gpu-specs/l4.c4091>

¹²<https://cloud.vast.ai/>

graph generation.

While *FGdiffusion* does not outperform state-of-the-art models on graph quality metrics (MMD and VUN) for a schema-less graph generation task, we observe that it demonstrates strong competitiveness on these metrics. This also highlights that, in its current development state, a key advantage of our approach is its efficiency and scalability. Indeed, thanks to complementary experiments¹³, we observe that while adjacency-based methods like DiGress are typically limited to graphs of up to 256 nodes, *FGdiffusion* can effectively scale to graphs with up to 3,000 nodes (assuming a mean degree of 10). This suggests that one of the next steps for improving our approach is addressing *graph size limitations*. We believe that increasing the training duration could enhance graph quality metrics. In this work, limited GPU resources prevented us from training for many steps, and it seems we undertrained the model compared to other works – within the information available from the related papers – possibly explaining the lower scores on the MMD and VUN metrics. Future improvements could also involve fine-tuning hyperparameters and exploring advanced techniques for handling larger graphs, such as hierarchical generation or distributed computing.

For the ontology-constrained graph generation task, we observe the effectiveness of *FGdiffusion* in producing KGs whose structure, and consequently semantics, tend to conform to a reference ontology. We acknowledge that the current set of constraints is limited, which may result in generating knowledge graphs that are not fully valid from the perspective of the VUN metric – similar to the Tree Graphs case – and consequently, it diminishes the apparent benefit of forcing, since VUN is a binary metric operating at the whole graph level (*i.e.* the larger the graphs, the higher the chance of breaking the metric). Nevertheless, the gains observed on the VUN metric through forcing reinforce our belief in the value of using a constrained inference approach rather than constrained learning for non-statistical constraints, particularly for reasons of flexibility (*e.g.* changing rules without retraining) and simplicity (*e.g.* challenges in implementing logical rules within the training loss).

This suggests a second next challenge for improving our approach is addressing *schema-related generative complexity*. Currently, our setup may struggle handling highly complex ontologies, such as those involving constructs like `rdfs:subClassOf`, `rdfs:subPropertyOf`, or `owl:intersectionOf`. While our method can enforce domain constraints, cardinality constraints, and functional property constraints, increased constraint complexity can impact the verification overhead during unmasking, especially as graph sizes grow. Consequently, future work could explore the integration of more advanced constraint satisfaction techniques – such as constraint programming or reinforcement learning – to better handle the expressivity of ontologies, while maintaining low overhead during the unmasking step.

Additionally, most of nodes in real-world knowledge graphs are more complex than just typed nodes: most of the data is composed of literals¹⁴ that could be string values or continuous values. Those type of data was not in the scope of this paper and should be explored in future work to account for ontologies implementing constructs such as `owl:DatatypeProperty`.

5. Conclusion

In this work, we focused on the ability of deep learning-based graph generation models to produce knowledge graphs that are both large-scale and representative of a specific business domain or a given situation. Current state of the art techniques show two main disadvantages: the limited size of models (due to algorithmic complexity related to memory and CPU/GPU constraints), which only allows the production of small to medium-sized graphs; the lack of guarantees regarding the shape and semantics of the generated graphs, which can lead to producing graphs that are not usable (*e.g.* non-compliant with a given ontology).

To overcome these limitations, we proposed and evaluated the *FGdiffusion* approach, which consists

¹³For both DiGress and *FGdiffusion* on the test machine described in Section 4.1, with comparable parameter sizes (approximately 10e6), we set the batch size to one and increased P_{max} until the training process failed.

¹⁴<https://www.w3.org/TR/rdf11-concepts/#section-Graph-Literal>

in improving deep learning graph generation through: 1) a specific encoding of data prior to the training phase; 2) the use of a diffusion-based model; 3) the application of constraints derived from reference ontologies to guide the generation process.

By comparing the performance of *FGdiffusion* to eight state-of-the-art models on a schema-less graph generation task, we observed that, while our method does not outperform all competitors in every quality metric of the generated graphs (MMD and VUN), it demonstrates strong competitiveness, thereby validating the effectiveness of the diffusion paradigm for graph synthesis. Extending this comparison to efficiency metrics, *FGdiffusion* is competitive with top autoregressive models like GEEL in inference time per TFLOPS and DLPerf, and significantly faster than diffusion baselines like DiGress, thanks to its discrete diffusion process. Testing *FGdiffusion* on three large-scale real-world KG datasets (ranging from 146K to 23M nodes) on an ontology-constrained graph generation task, we observed the ability of our approach to produce KGs whose semantics adhere to a reference ontology. The graph quality gains (VUN metric) we observed also reinforce our belief in the value of using a constrained inference approach rather than constrained learning for non-statistical constraints, particularly for reasons of flexibility (e.g. changing rules without retraining) and simplicity (e.g. challenges in implementing logical rules within the training loss).

Overall, our experiments confirm that combining flattened graph representations, diffusion modeling, and guided unmasking effectively tackles scaling and domain relevance in KG generation. However, to enhance practicality and real-world applicability, it remains essential to address certain limitations of our approach. We envision future work along two main axes: *Optimizing the model performance at the training stage*, including exploring the effect of increasing training duration, as well as investigating advanced techniques for handling larger graphs – such as guided subgraph sampling, hierarchical generation, or distributed computing approaches; and *Improving constrained-inference*, including gradually adding additional constraints, comparing the quality of generated graphs relative to the expressivity of the ontologies used, and exploring how to incorporate literals through multimodal diffusion.

To conclude, given the difficulty in benchmarking the training phase of our approach due to the lack of consistent information in related work, we also advocate for the *standardization of compute KPIs in AI research*. Explicitly reporting metrics such as FLOPs, energy consumption, and hardware utilization during training and inference would improve transparency, reproducibility, and fair comparison – especially as resource-efficient AI becomes increasingly important. Such standards would allow the community to better assess model efficiency and practicality, promoting innovations that are accessible and sustainable.

Acknowledgments

We would like to thank Philippe DOOZE (Orange) and Ovidiu PASCAL (Orange) for their active participation in the brainstorming group on graph generation, within which the *FGdiffusion* project was situated.

Declaration on Generative AI

During the preparation of this work, the authors used GPT-4.1-nano to assist with *grammar and spelling checks*. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the publication’s content.

References

- [1] A. Hogan, E. Blomqvist, M. Cochez, C. D’amato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, Antoine Zimmermann, Knowledge Graphs, ACM Comput. Surv. 54 (2021). doi:10.1145/3447772.

- [2] C. Vignac, I. Krawczuk, A. Siraudin, B. Wang, V. Cevher, P. Frossard, DiGress: Discrete Denoising diffusion for graph generation, 2023. URL: <https://arxiv.org/abs/2209.14734>. arXiv:2209.14734.
- [3] H. Huang, L. Sun, B. Du, Y. Fu, W. Lv, GraphGDP: Generative Diffusion Processes for Permutation Invariant Graph Generation, 2022. URL: <https://arxiv.org/abs/2212.01842>. arXiv:2212.01842.
- [4] A. Lou, C. Meng, S. Ermon, Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution, 2024. URL: <https://arxiv.org/abs/2310.16834>. arXiv:2310.16834.
- [5] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, R. van den Berg, Structured Denoising Diffusion Models in Discrete State-Spaces, 2023. URL: <https://arxiv.org/abs/2107.03006>. arXiv:2107.03006.
- [6] T. R. Gruber, Toward Principles for the Design of Ontologies Used for Knowledge Sharing?, *International Journal of Human-Computer Studies* 43 (1995) 907–928. doi:10.1006/ijhc.1995.1081.
- [7] H. Li, R. Zhang, L. Wang, B. Yu, Y. Wang, Y. Wei, K. Wang, R. Y. D. Xu, B. Wang, Deep Sparse Latent Feature Models for Knowledge Graph Completion, 2024. URL: <https://arxiv.org/abs/2411.15694>. arXiv:2411.15694.
- [8] L. Luo, Z. Zhao, G. Haffari, D. Phung, C. Gong, S. Pan, GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation, 2025. URL: <https://arxiv.org/abs/2502.01113>. arXiv:2502.01113.
- [9] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, X. Tang, Q. He, Z. Hua, B. Long, T. Zhao, N. Shah, A. Javari, Y. Xia, J. Tang, Retrieval-Augmented Generation with Graphs (GraphRAG), 2025. URL: <https://arxiv.org/abs/2501.00309>. arXiv:2501.00309.
- [10] Y. Zhu, Y. Du, Y. Wang, Y. Xu, J. Zhang, Q. Liu, S. Wu, A Survey on Deep Graph Generation: Methods and Applications, 2022. URL: <https://arxiv.org/abs/2203.06714>. arXiv:2203.06714.
- [11] J. You, R. Ying, X. Ren, W. L. Hamilton, J. Leskovec, GraphRNN: Generating Realistic Graphs with Deep Auto-regressive Models, 2018. URL: <https://arxiv.org/abs/1802.08773>. arXiv:1802.08773.
- [12] X. Chen, Y. Wang, J. He, Y. Du, S. Hassoun, X. Xu, L.-P. Liu, Graph Generative Pre-trained Transformer, 2025. URL: <https://arxiv.org/abs/2501.01073>. arXiv:2501.01073.
- [13] J. Ho, A. Jain, P. Abbeel, Denoising Diffusion Probabilistic Models, 2020. URL: <https://arxiv.org/abs/2006.11239>. arXiv:2006.11239.
- [14] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, T.-Y. Liu, Do Transformers Really Perform Bad for Graph Representation?, 2021. URL: <https://arxiv.org/abs/2106.05234>. arXiv:2106.05234.
- [15] S. Geisler, Y. Li, D. Mankowitz, A. T. Cemgil, S. Günnemann, C. Paduraru, Transformers Meet Directed Graphs, 2023. URL: <https://arxiv.org/abs/2302.00049>. arXiv:2302.00049.
- [16] J. K. Christopher, S. Baek, F. Fioretto, Constrained Synthesis with Projected Diffusion Models, 2024. URL: <https://arxiv.org/abs/2402.03559>. arXiv:2402.03559.
- [17] B. T. Willard, R. Louf, Efficient Guided Generation for LLMs, arXiv preprint arXiv:2307.09702 (2023).
- [18] A. Li, R. Beeson, Aligning Diffusion Model with Problem Constraints for Trajectory Optimization, 2025. URL: <https://arxiv.org/abs/2504.00342>. arXiv:2504.00342.
- [19] J.-H. Bastek, W. Sun, D. M. Kochmann, Physics-Informed Diffusion Models, 2025. URL: <https://arxiv.org/abs/2403.14404>. arXiv:2403.14404.
- [20] A. Berzins, A. Radler, E. Volkmann, S. Sanokowski, S. Hochreiter, J. Brandstetter, Geometry-Informed Neural Networks, 2024. URL: <https://arxiv.org/abs/2402.14009>. arXiv:2402.14009.
- [21] L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui, M.-H. Yang, Diffusion Models: A Comprehensive Survey of Methods and Applications, 2024. URL: <https://arxiv.org/abs/2209.00796>. arXiv:2209.00796.
- [22] Chenshuang Zhang, Chaoning Zhang, Mengchun Zhang, In So Kweon, Junmo Kim, Text-to-image Diffusion Models in Generative AI: A Survey, 2024. URL: <https://arxiv.org/abs/2303.07909>. arXiv:2303.07909.
- [23] Zeyue Tian, Yizhu Jin, Zhaoyang Liu, Ruibin Yuan, Xu Tan, Qifeng Chen, Wei Xue, Yike Guo, AudioX: Diffusion Transformer for Anything-to-Audio Generation, 2025. URL: <https://arxiv.org/abs/2503.10522>. arXiv:2503.10522.
- [24] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, C. Li, Large Language

- Diffusion Models, 2025. URL: <https://arxiv.org/abs/2502.09992>. arXiv:2502.09992.
- [25] Richard Cyganiak, David Wood, Markus Lanthaler, RDF 1.1 Concepts and Abstract Syntax, W3C Recommendation, W3C, 2014. URL: <http://www.w3.org/TR/rdf11-concepts/>.
 - [26] Dan Brickley, R.V. Guha, RDF Schema 1.1, W3C Recommendation, W3C, 2014. URL: <https://www.w3.org/TR/rdf-schema/>.
 - [27] W3C OWL Working Group, OWL 2 Web Ontology Language, W3C Recommendation, W3C, 2012. URL: <https://www.w3.org/TR/owl2-overview/>.
 - [28] G. Abuoda, D. Dell’Agllo, A. Keen, K. Hose, Transforming RDF-star to Property Graphs: A Preliminary Analysis of Transformation Approaches – Extended Version, 2022. doi:10.48550/arXiv.2210.05781. arXiv:2210.05781.
 - [29] O. Hartig, Reconciliation of RDF* and Property Graphs, 2014. URL: <https://arxiv.org/abs/1409.3288>. arXiv:1409.3288.
 - [30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention Is All You Need, 2023. URL: <https://arxiv.org/abs/1706.03762>. arXiv:1706.03762.
 - [31] D. Reynolds, The Organization Ontology, W3C Recommendation, W3C, 2014. URL: <https://www.w3.org/TR/vocab-org/>.
 - [32] K. Frans, D. Hafner, S. Levine, P. Abbeel, One Step Diffusion via Shortcut Models, 2025. URL: <https://arxiv.org/abs/2410.12557>. arXiv:2410.12557.
 - [33] Yang Song, Prafulla Dhariwal, Mark Chen, Ilya Sutskever, Consistency Models, 2023. URL: <https://arxiv.org/abs/2303.01469>. arXiv:2303.01469.
 - [34] K. Martinkus, A. Loukas, N. Perraudin, R. Wattenhofer, SPECTRE: Spectral Conditioning Helps to Overcome the Expressivity Limits of One-shot Graph Generators, 2022. URL: <https://arxiv.org/abs/2204.01613>. arXiv:2204.01613.
 - [35] NASA Goddard Earth Sciences Data and Information Services Center (GES-DISC), nasa-eo-knowledge-graph, 2024. URL: <https://huggingface.co/datasets/nasa-gesdisc/nasa-eo-knowledge-graph>. doi:10.57967/hf/3463.
 - [36] Arnab Sinha, Zhihong Shen, Yang Song, Hao Ma, Darrin Eide, Bo-June (Paul) Hsu, Kuansan Wang, An Overview of Microsoft Academic Service (MAS) and Applications, in: Proceedings of the 24th International Conference on World Wide Web, WWW ’15 Companion, Association for Computing Machinery, New York, NY, USA, 2015, p. 243–246. URL: <https://doi.org/10.1145/2740908.2742839>. doi:10.1145/2740908.2742839.
 - [37] Danilo Dessì, Francesco Osborne, Davide Buscaldi, Diego Reforgiato Recupero, Enrico Motta, CS-KG 2.0: A Large-scale Knowledge Graph of Computer Science, Scientific Data 12 (2025) 964. doi:10.1038/s41597-025-05200-8.
 - [38] David Beckett, Tim Berners-Lee, Eric Prud’hommeaux, Gavin Carothers, RDF 1.1 Turtle: Terse RDF Triple Language, W3C Recommendation, W3C, 2017. URL: <https://www.w3.org/TR/turtle/>.
 - [39] R. Liao, Y. Li, Y. Song, S. Wang, C. Nash, W. L. Hamilton, D. Duvenaud, R. Urtasun, R. S. Zemel, Efficient Graph Generation with Graph Recurrent Attention Networks, 2020. URL: <https://arxiv.org/abs/1910.00760>. arXiv:1910.00760.
 - [40] Yunhui Jang, Seul Lee, Sungsoo Ahn, A Simple and Scalable Representation for Graph Generation, 2024. URL: <https://arxiv.org/abs/2312.02230>. arXiv:2312.02230.
 - [41] Hanjun Dai, Azade Nazi, Yujia Li, Bo Dai, Dale Schuurmans, Scalable Deep Generative Modeling for Sparse Graphs, 2020. URL: <https://arxiv.org/abs/2006.15502>. arXiv:2006.15502.
 - [42] Andreas Bergmeister, Karolis Martinkus, Nathanaël Perraudin, Roger Wattenhofer, Efficient and Scalable Graph Generation through Iterative Local Expansion, 2024. URL: <https://arxiv.org/abs/2312.11529>. arXiv:2312.11529.
 - [43] Nathaniel Diamant, Alex M. Tseng, Kangway V. Chuang, Tommaso Biancalani, Gabriele Scalia, Improving Graph Generation by Restricting Graph Bandwidth, 2023. URL: <https://arxiv.org/abs/2301.10857>. arXiv:2301.10857.
 - [44] Y. Qin, M. Madeira, D. Thanou, P. Frossard, DeFoG: Discrete Flow Matching for Graph Generation, 2025. URL: <https://arxiv.org/abs/2410.04263>. arXiv:2410.04263.
 - [45] OneFlow Inc., DLPerf – Deep Learning Framework Performance Profiling Toolkit, <https://github.com>.

com/Oneflow-Inc/DLPerf/, 2020.

- [46] J. Jo, S. Lee, S. J. Hwang, Score-based generative modeling of graphs via the system of stochastic differential equations, in: Proceedings of the 39th International Conference on Machine Learning, volume 162 of *Proceedings of Machine Learning Research*, PMLR, ????, pp. 10362–10383. URL: <https://proceedings.mlr.press/v162/jo22a.html>.